Coding: API Design Standards

APIDesign

```
---
applyTo: '**/*.cs'
---

# API Design Best Practices

API Design Best Practices that AI should follow.

## Table of Contents

1. [Introduction](#introduction)
2. [RESTful Web API Design Concepts](#restful-web-api-design-concepts)
3. [Resource URI Design](#resource-uri-design)
4. [HTTP Methods and Status Codes](#http-methods-and-status-codes)
5. [Resource Representation and MIME Types](#resource-representation-and-mime-types)
6. [Asynchronous Operations](#asynchronous-operations)
7. [Data Pagination and Filtering](#data-pagination-and-filtering)
8. [Partial Responses](#partial-responses)
9. [HATEOAS Implementation](#hateoas-implementation)
10. [API Versioning Strategies](#api-versioning-strategies)
11. [Multitenant API Design](#multitenant-api-design)
12. [Distributed Tracing](#distributed-tracing)
13. [API Maturity Model](#api-maturity-model)
14. [OpenAPI Initiative](#openapi-initiative)
15. [Quick Reference](#quick-reference)

---

## Introduction

A RESTful web API implementation employs Representational State Transfer (REST) architectural principles to achieve a stateless, loosely coupled interface between a client and service. This knowledge base summarizes Microsoft Azure's best practices for designing RESTful web APIs.

### Core Principles

- **Platform Independence**: Clients can call the web API regardless of internal implementation
- **Loose Coupling**: Client and web service can evolve independently
- **Standard Protocols**: Use HTTP as the standard protocol with clear documentation
- **Familiar Data Formats**: Support JSON or XML for data exchange

---

## RESTful Web API Design Concepts

### Key Concepts

#### Uniform Resource Identifier (URI)

REST APIs are designed around resources - any kind of object, data, or service that the client can access. Each resource is represented by a URI that uniquely identifies that resource.

**Example:**

```http
https://api.contoso.com/orders/1
```

Resource Representation

Defines how a resource identified by its URI is encoded and transported over HTTP in a specific format (XML or JSON).

Example Response:

```json
{ "orderId": 1, "orderValue": 99.9, "productId": 1, "quantity": 1 }
```

Uniform Interface

Achieves loose coupling between client and service implementations using standard HTTP verbs: `GET`, `POST`, `PUT`, `PATCH`, and `DELETE`.

Stateless Request Model
```

HTTP requests are independent and can occur in any order. No transient state information is kept between requests.

Hypermedia Links

REST APIs can be driven by hypermedia links contained in each resource representation.

Example:

```json
{
  "orderID": 3,
  "productID": 2,
  "quantity": 4,
  "orderValue": 16.6,
  "links": [
    {
      "rel": "product",
      "href": "https://api.contoso.com/customers/3",
      "action": "GET"
    },
    {
      "rel": "product",
      "href": "https://api.contoso.com/customers/3",
      "action": "PUT"
    }
  ]
}
```

Resource URI Design

Best Practices

Use Nouns, Not Verbs

Base resource URIs on nouns (the resource) and not verbs (the operations on the resource).

Good:

```http
https://api.contoso.com/orders
```

Avoid:

```http
https://api.contoso.com/create-order
```

URI Naming Conventions

1. **Use nouns for resource names**

   - Use `/orders` instead of `/create-order`
   - HTTP methods already imply the verbal action

2. **Use plural nouns for collection URIs**

   - `/customers` for the customer collection
   - `/customers/5` for customer with ID 5

3. **Consider relationships between resources**

   - `/customers/5/orders` for all orders for customer 5
   - `/orders/99/customer` for the customer of order 99

4. **Keep relationships simple and flexible**

   - Avoid complex URIs like `/customers/1/orders/99/products`
   - Prefer simpler patterns: `/customers/1/orders` then `/orders/99/products`

5. **Enforce kebab casing for multi-word resource names**

   - Use `/customer-orders` instead of `/customerOrders`

6. **Avoid large numbers of small resources**

   - Prevent "chatty" APIs that require multiple requests
   - Balance against fetching unnecessary data

7. **Don't mirror database structure**
   - Think of the API as an abstraction of the database
   - Introduce mapping layers when necessary

Non-Resource Operations

For operations that don't map to specific resources, use pseudo resources with query strings:

```http
GET /add?operand1=99&operand2=1
```

Use this pattern sparingly.

HTTP Methods and Status Codes

HTTP Methods Overview

Resource	POST	GET	PUT	DELETE
/customers	Create a new customer	Retrieve all customers	Bulk update of customers	Remove all customers
/customers/1	Error	Retrieve customer 1 details	Update customer 1 if exists	Remove customer 1
/customers/1/orders	Create new order for customer 1	Retrieve all orders for customer 1	Bulk update orders for customer 1	Remove all orders for customer 1

GET Requests

Retrieves a representation of the resource at the specified URI.

Status Codes:

- `200 (OK)`: Successfully returned the resource
- `204 (No Content)`: Response body contains no content
- `404 (Not Found)`: Requested resource can't be found

POST Requests

Creates a resource. The server assigns a URI for the new resource. Based on the logic of the endpoint,
if API is purely creating a resource, returning HTTP status code `201` must be enforced.

Important: Clients shouldn't attempt to create their own URI. Submit requests to the collection URI.

Status Codes:

- `200 (OK)`: Processing done but no new resource created
- `201 (Created)`: Resource created successfully (include Location header)
- `202 (Accepted)`: Request accepted for processing but not completed
- `204 (No Content)`: Response body contains no content
- `400 (Bad Request)`: Invalid data in request
- `405 (Method Not Allowed)`: POST not supported on this URI

PUT Requests

Updates an existing resource or creates a new resource if it doesn't exist.

Key Points:

- Must be idempotent (same request multiple times = same result)
- Applied to individual items, not collections
- Client specifies complete representation of the resource

Status Codes:

- `200 (OK)`: Resource updated successfully
- `201 (Created)`: Resource created successfully
- `204 (No Content)`: Updated successfully, no content in response
- `409 (Conflict)`: Conflict with current resource state

PATCH Requests

Performs partial update to an existing resource. More efficient than PUT as only changes are sent.

JSON Merge Patch Example

Original Resource:

```json
```

```
{
  "name": "gizmo",
  "category": "widgets",
  "color": "blue",
  "price": 10
}
```

Merge Patch:

```json
{
  "price": 12,
  "color": null,
  "size": "small"
}
```

Result: Updates price, deletes color, adds size.

Media Types:

- JSON Merge Patch: `application/merge-patch+json`
- JSON Patch: `application/json-patch+json`

Status Codes:

- `200 (OK)`: Resource updated successfully
- `400 (Bad Request)`: Malformed patch document
- `409 (Conflict)`: Changes can't be applied to current state
- `415 (Unsupported Media Type)`: Patch format not supported

DELETE Requests

Removes the resource at the specified URI.

Status Codes:

- `204 (No Content)`: Resource deleted successfully
- `404 (Not Found)`: Resource doesn't exist

Resource Representation and MIME Types

Media Types

For non-binary data, most web APIs support:

- **JSON**: `application/json` (recommended)
- **XML**: `application/xml`

Content-Type Header

Specifies the resource representation format in requests:

```http
POST https://api.contoso.com/orders
Content-Type: application/json; charset=utf-8
Content-Length: 57

{"Id":1,"Name":"Gizmo","Category":"Widgets","Price":1.99}
```

Accept Header

Client specifies acceptable response formats:

```http
GET https://api.contoso.com/orders/2
Accept: application/json, application/xml
```

Status Codes:

- `415 (Unsupported Media Type)`: Server doesn't support the media type
- `406 (Not Acceptable)`: Server can't match any listed media types

Asynchronous Operations

For long-running operations, implement asynchronous processing to avoid unacceptable latency.

Implementation Pattern

1. **Return 202 (Accepted)** with Location header pointing to status endpoint:

```http
HTTP/1.1 202 Accepted
Location: /api/status/12345
```

2. **Status endpoint** returns current status:

```http
HTTP/1.1 200 OK
Content-Type: application/json

{
    "status":"In progress",
    "link": { "rel":"cancel", "method":"delete", "href":"/api/status/12345" }
}
```

3. **Completion** returns 303 (See Other) with Location of new resource:

```http
HTTP/1.1 303 See Other
Location: /api/orders/12345
```

Data Pagination and Filtering

Pagination

Divide large datasets into manageable chunks using query parameters:

```http
GET /orders?limit=25&offset=50
```

Parameters:

- `limit`: Maximum number of items to return (consider max limits for security)
- `offset`: Starting index for the data

Best Practices:

- Provide meaningful defaults (e.g., `limit=25`, `offset=0`)
- Impose upper limits to prevent denial-of-service attacks

Filtering

Allow clients to refine datasets by applying conditions:

```http
GET /orders?minCost=100&status=shipped
```

Additional Features

Sorting

```http
GET /orders?sort=price
```

Note: Sorting can negatively affect caching as query parameters form part of cache keys.

Field Selection

```http
GET /orders?fields=ProductID,Quantity
```

Important: Validate requested fields to ensure client access permissions and prevent exposure of internal fields.

Partial Responses

Support partial retrieval of large binary resources to overcome connection issues and improve response times.

Implementation

1. **Support Accept-Ranges header** for GET requests:

```http
HEAD https://api.contoso.com/products/10?fields=productImage
```

2. **Response indicates partial support**:

```http
HTTP/1.1 200 OK
Accept-Ranges: bytes
Content-Type: image/jpeg
Content-Length: 4580
```

3. **Client requests partial content**:

```http
GET https://api.contoso.com/products/10?fields=productImage
Range: bytes=0-2499
```

4. **Server returns partial content**:

```http
HTTP/1.1 206 Partial Content
Accept-Ranges: bytes
Content-Type: image/jpeg
Content-Length: 2500
Content-Range: bytes 0-2499/4580

[...]
```

HATEOAS Implementation

Hypertext as the Engine of Application State (HATEOAS) enables navigation through resources without prior knowledge of the URI schema.

Example Implementation

```json
{
  "orderID": 3,
  "productID": 2,
  "quantity": 4,
  "orderValue": 16.6,
  "links": [
    {
      "rel": "customer",
      "href": "https://api.contoso.com/customers/3",
      "action": "GET",
      "types": ["text/xml", "application/json"]
    },
    {
      "rel": "customer",
      "href": "https://api.contoso.com/customers/3",
      "action": "PUT",
      "types": ["application/x-www-form-urlencoded"]
    },
    {
      "rel": "self",
      "href": "https://api.contoso.com/orders/3",
      "action": "GET",
      "types": ["text/xml", "application/json"]
    }
  ]
}
```

Key Elements

- **rel**: Relationship type ("customer", "self")
- **href**: URI for the operation
- **action**: HTTP method
- **types**: Supported MIME types

Note: No general-purpose standards exist for HATEOAS modeling. The set of links can change based on resource state.

API Versioning Strategies

API versioning is optional but we would recommend header versioning or media type versioning. Consider API versioning in cases where we expect frequent significant breaking changes to the API payloads or responses.

1. No Versioning

Simplest approach for internal APIs. Adding content to existing resources might not break clients if they ignore unknown fields.

Example Evolution:

```json
// Version 1
{"id":3,"name":"Fabrikam, Inc.","address":"1 Microsoft Way Redmond WA 98053"}

// Version 2 (adds dateCreated)
{"id":3,"name":"Fabrikam, Inc.","dateCreated":"2025-03-22T12:11:38.0376089Z","address":"1 Microsoft Way Redmond WA 98053"}
```


2. Header Versioning (Optional)

If versioning is required, use custom header to indicate version:

```http
GET https://api.contoso.com/customers/3
X-Custom-Header: api-version=2
```

Pros: Clean URIs
Cons: Requires custom header handling, complicates HATEOAS

Caching Considerations
```http
GET https://api.contoso.com/customers/3
Custom-Header: api-version=2
```

Pros: Clean URIs
Cons: Requires custom header handling, complicates HATEOAS

Caching Considerations
```http
GET https://api.contoso.com/customers/3
Custom-Header: api-version=2
```

Pros: Clean URIs
Cons: Requires custom header handling, complicates HATEOAS

Caching Considerations

- **URI and Query String versioning**: Cache-friendly (same URI/query = same data)
- **Header and Media Type versioning**: Require more logic, can result in cache duplication

---

Distributed Tracing

Enable end-to-end visibility in distributed systems using trace context headers.

Implementation

Use headers like `X-Correlation-ID`, `X-Request-ID`, or `X-Trace-ID`:

Request:

```http
GET https://api.contoso.com/orders/3
Correlation-ID: 0f8fad5b-d9cb-469f-a165-70867728950e
```

Response:

```http
HTTP/1.1 200 OK
Correlation-ID: 0f8fad5b-d9cb-469f-a165-70867728950e

{...}
```

```

### Benefits

- Seamless request tracking across services
- Rapid failure identification
- Latency monitoring
- API dependency mapping
- Enhanced debugging capabilities

---

## API Maturity Model

The Richardson Maturity Model (RMM) defines four levels of REST API maturity:

### Level 0: Single URI, POST Operations

- Define one URI
- All operations are POST requests
- Example: SOAP web services

### Level 1: Resource-Based URIs

- Create separate URIs for individual resources
- Not yet RESTful but begins alignment with REST principles

### Level 2: HTTP Methods

- Use HTTP methods to define operations on resources
- Most published web APIs align with this level

### Level 3: Hypermedia (HATEOAS)

- Use hypermedia for application state navigation
- Truly RESTful according to Fielding's definition

---

## OpenAPI Initiative

The OpenAPI Specification (OAS) standardizes REST API descriptions across vendors.

### Key Considerations

1. **Opinionated Guidelines**: OAS provides specific guidelines for REST API design that benefit interoperability but require conformance

2. **Contract-First Approach**: Design the API contract (interface) first, then implement the code

3. **Tool Ecosystem**: Tools like Swagger can generate client libraries and documentation from API contracts

### Benefits

- Standardized API documentation
- Automated client library generation
- Improved interoperability
- Contract-driven development

---

## Quick Reference

### HTTP Status Codes Summary

| Code | Method                  | Meaning                |
| ---- | ----------------------- | ---------------------- |
| 200  | GET, PUT, PATCH         | Success                |
| 201  | POST, PUT               | Created                |
| 204  | GET, PUT, PATCH, DELETE | No Content             |
| 202  | POST                    | Accepted               |
| 400  | POST, PATCH             | Bad Request            |
| 404  | GET, DELETE             | Not Found              |
| 405  | POST                    | Method Not Allowed     |
| 406  | GET                     | Not Acceptable         |
| 409  | PUT, PATCH              | Conflict               |
| 415  | PATCH                   | Unsupported Media Type |

### URI Design Checklist

- ✅ Use nouns, not verbs
- ✅ Use plural nouns for collections
- ✅ Keep relationships simple (avoid deep nesting)
```

- ✅ Avoid mirroring database structure
- ✅ Consider resource granularity (avoid chatty APIs)

Versioning Strategy Comparison

Strategy	Pros	Cons	Cache-Friendly
No Versioning	Simple	Breaking changes	✅
URI Versioning	Clear, simple	Unwieldy over time	✅
Query String	Same URI for resource	Browser cache issues	⚠️
Header Versioning	Clean URIs	Complex logic	❌
Media Type	HTTP standard, HATEOAS-friendly	Complex implementation	❌

Security Considerations

- Validate all input parameters
- Implement rate limiting for pagination
- Use HTTPS for all communications
- Validate field selection permissions
- Implement proper authentication and authorization
- Consider tenant isolation in multitenant scenarios

This knowledge base is based on Microsoft Azure's API Design Best Practices documentation and provides a comprehensive reference for designing RESTful web APIs.